

Réalisation professionnelle : Recensement d'un parc informatique avec GLPI

Compétences mis en oeuvres

C1 - Recensement et identification des ressources numériques

Contexte

Dans le cadre de mon stage réalisé à l'Efrei, j'ai été amené à recenser et identifier les ressources numériques d'une organisation. Pour cela, j'ai déployé GLPI sur un serveur Azure (VM Debian/Ubuntu, IP : 51.103.220.166) afin de centraliser l'inventaire du parc informatique.

Démarche suivi

Étape 1 — Déploiement du serveur GLPI sur Azure

J'ai d'abord provisionné une machine virtuelle sur Azure (Debian/Ubuntu) et installé GLPI 10.0.20 via le script `install-glpi.sh`. Ce script automatise l'ensemble de la chaîne : Apache, PHP 8.3 et MariaDB.

Étape 2 — Déploiement de l'agent GLPI sur les postes clients

Une fois le serveur opérationnel, j'ai installé l'agent GLPI sur chaque poste client à l'aide du script `install-glpiagent.sh`. L'agent remonte automatiquement l'inventaire matériel et logiciel vers le serveur.

Script complet — `install-glpiagent.sh` :

```
#!/bin/bash

#####
#   Launch with SUDO   #
#####

# Arrêt du script en cas d'erreur
set -e

# Installation des dépendances Perl nécessaires à l'agent
sudo apt install -y perl libnet-ssleay-perl libcrypt-ssleay-perl \
  libio-socket-ssl-perl libarchive-zip-perl libcompress-zlib-perl \
  libwww-perl libxml-xpath-perl libyaml-tiny-perl \
  dmidecode hwdmdata pciutils usbutils

# Téléchargement du paquet d'installation de l'agent GLPI 1.7
wget https://github.com/glpi-project/glpi-agent/releases/download/1.7/glpi-agent-1.7-linux-
installer.pl

# Installation de l'agent avec connexion automatique au serveur GLPI
sudo perl glpi-agent-1.7-linux-installer.pl \
  --server="http://51.103.220.166/front/inventory.php" \
  --install --runnow
```

```
# Vérification que le service glpi-agent est bien actif
systemctl status glpi-agent

# Forcer un inventaire immédiat vers le serveur
glpi-agent --force
```

Étape 3 — Consultation et export de l'inventaire

Les données remontées par les agents ont permis d'obtenir un inventaire complet accessible depuis l'interface GLPI. J'ai exporté trois rapports PDF comme preuves :

- export_orde.pdf : liste des ordinateurs recensés (modèle, IP, OS)
- export_soft.pdf : logiciels installés sur chaque poste
- export_user.pdf : comptes utilisateurs créés et associés aux équipements

Conclusion

Cette réalisation m'a permis de comprendre comment recenser et identifier les ressources numériques d'une organisation à l'aide d'un outil ITSM. J'ai appris à déployer un agent sur des machines Linux et à exploiter les données remontées pour constituer un inventaire fiable et centralisé du système d'information.

Réalisation professionnelle : Mise en place d'un système de tickets avec GLPI

Compétences mis en oeuvres

C7 - Collecte, suivi et orientation des demandes

Contexte

Dans le cadre de mon stage, j'ai configuré le module de gestion des demandes de GLPI afin de permettre aux utilisateurs de soumettre des tickets d'incidents ou de demandes, et d'en assurer le suivi jusqu'à leur résolution.

Démarche suivi

Étape 1 — Création des comptes utilisateurs

J'ai créé les comptes utilisateurs dans GLPI et défini les groupes de techniciens responsables du traitement des tickets. Chaque utilisateur se voit attribuer un profil (Observateur, Technicien ou Super-Admin).

```
# Accès à l'interface GLPI
http://51.103.220.166

# Navigation : Administration > Utilisateurs > Ajouter
# Champs renseignés :
#   - Nom, prénom, identifiant
#   - Groupe d'appartenance
#   - Profil : Observateur / Technicien / Super-Admin
```

Étape 2 — Configuration des catégories et règles d'attribution

J'ai paramétré les catégories de tickets (incident matériel, logiciel, réseau...) et configuré des règles d'attribution automatique pour orienter chaque ticket vers le bon groupe de techniciens.

Étape 3 — Cycle de vie d'un ticket

Les utilisateurs peuvent ouvrir un ticket depuis leur navigateur. Chaque ticket passe par les statuts suivants :

Nouveau -> En cours (attribué) -> En cours (planifié) -> Résolu -> Clos

L'historique complet des interventions est conservé dans GLPI et exporté dans export_user.pdf.

Conclusion

Cette réalisation m'a permis de mettre en place un processus structuré de collecte et de suivi des demandes. J'ai compris l'importance d'orienter les tickets vers les bonnes personnes et de maintenir une traçabilité complète des interventions pour améliorer la qualité de service.

Réalisation professionnelle : Référencement d'un service en ligne et mesure de sa visibilité

Compétences mis en oeuvres

C11 - Référencement des services en ligne de l'organisation et mesure de leur visibilité

Contexte

Dans le cadre du projet, j'ai travaillé sur la visibilité en ligne d'un service web déployé sur GitHub Pages. J'ai intégré des balises HTML sémantiques pour améliorer le référencement naturel et mis en place une balise Google Tag pour mesurer l'audience et la visibilité du service.

Démarche suivi

Étape 1 — Structuration HTML sémantique et balises méta SEO

J'ai structuré le code HTML avec des balises sémantiques (header, main, section, footer) et des métadonnées pour optimiser l'indexation par les moteurs de recherche.

```
<head>
  <meta charset="UTF-8">
  <meta name="description"
    content="Erreur 404 Found - Hackathon BTS SIO 2025">
  <meta name="keywords"
    content="BTS SIO, hackathon, GLPI, informatique">
  <title>Erreur 404 Found | Hackathon 2025</title>
</head>
```

Étape 2 — Intégration de la balise Google Tag (gtag.js)

J'ai intégré le script Google Tag dans le <head> de chaque page pour activer le suivi de l'audience. Cela permet de mesurer le trafic, les sources de visites et les pages consultées.

```
<!-- Google tag (gtag.js) -->
<script async
  src="https://www.googletagmanager.com/gtag/js?id=G-XXXXXXXXXX">
</script>
<script>
  window.dataLayer = window.dataLayer || [];
  function gtag(){dataLayer.push(arguments);}
  gtag('js', new Date());
  gtag('config', 'G-XXXXXXXXXX');
</script>
```

Étape 3 — Déploiement et mesure de visibilité

Le site a été déployé sur GitHub Pages avec une URL publique permanente. La structure H1/H2 respectée et les métadonnées renseignées permettent une indexation correcte. Les données Google Tag permettent de mesurer en temps réel l'audience et la provenance des visiteurs.

Conclusion

Cette réalisation m'a permis de comprendre les mécanismes du référencement naturel (SEO) et d'apprendre à mesurer la visibilité d'un service en ligne. L'intégration de Google Tag m'a donné accès à des données d'audience concrètes, utiles pour évaluer et améliorer la présence en ligne d'une organisation.

Réalisation professionnelle : Planification du déploiement de l'infrastructure GLPI

Compétences mis en oeuvres

C14 - Planification des activités

Contexte

Dans le cadre de mon stage, j'ai dû organiser et planifier l'ensemble des activités liées au déploiement de l'infrastructure GLPI sur Azure, en tenant compte des contraintes techniques et des délais imposés.

Démarche suivie

Étape 1 — Découpage du projet en phases

J'ai défini 4 phases principales couvrant l'ensemble du cycle de déploiement, de l'analyse initiale à la mise en production.

```
Phase 1 – Analyse et recensement des besoins
-> Identification des ressources nécessaires
-> Choix des technologies (Azure, GLPI, GitHub)

Phase 2 – Installation et configuration
-> Déploiement de la VM Azure (Debian/Ubuntu)
-> Installation GLPI 10.0.20 + Apache + PHP 8.3 + MariaDB
-> Configuration du VirtualHost et des permissions

Phase 3 – Déploiement des agents et tests
-> Installation de l'agent GLPI sur les postes clients
-> Tests de remontée d'inventaire
-> Création des comptes utilisateurs et tickets de test

Phase 4 – Mise en production et sauvegarde
-> Activation des sauvegardes automatiques (cron)
-> Transfert vers Google Drive via rclone
-> Validation finale et livraison
```

Étape 2 — Suivi avec Trello (tableau Kanban)

Chaque tâche a été créée dans Trello avec les colonnes "À faire", "En cours" et "Terminé". Cela m'a permis de visualiser l'avancement global, de prioriser les actions critiques et de coordonner les différentes interventions.

Conclusion

Cette réalisation m'a permis de structurer mon travail et de respecter les délais imposés grâce à une planification rigoureuse. L'utilisation de Trello m'a aidé à maintenir une vue d'ensemble du projet et à coordonner les différentes tâches de manière efficace.

Réalisation professionnelle : Déploiement d'un service GLPI sur Azure

Compétences mis en oeuvres

C17 - Déploiement d'un service

Contexte

Dans le cadre de mon stage, j'ai déployé une infrastructure GLPI complète sur un serveur Azure (VM Debian/Ubuntu, IP : 51.103.220.166). Ce déploiement comprenait l'installation automatisée de GLPI, la mise en place d'un agent d'inventaire, ainsi qu'un système de sauvegarde et restauration automatisé vers Google Drive.

Démarche suivie

Étape 1 — Installation automatisée de GLPI (install-glpi.sh)

J'ai créé un script bash complet qui automatise l'installation de toute la pile LAMP (Apache, PHP 8.3, MariaDB) ainsi que GLPI 10.0.20. Le script détecte automatiquement l'IP du serveur, configure le VirtualHost Apache et sécurise les cookies PHP.

```
#!/bin/bash
set -e # Arrêt du script en cas d'erreur

# Étape 1 : Récupération de l'adresse IP de la machine
SERVER_IP=$(hostname -I | awk '{print $1}')
echo "L'adresse IP détectée est : $SERVER_IP"

# Étape 2 : Mise à jour système + installation Apache
apt update -y
apt install apache2 -y

# Étape 3 : Ajout du dépôt PHP 8.3 (sury.org) et installation PHP
apt install -y lsb-release ca-certificates apt-transport-https curl
curl -sSLo /tmp/debsuryorg-archive-keyring.deb https://packages.sury.org/debsuryorg-archive-keyring.deb
dpkg -i /tmp/debsuryorg-archive-keyring.deb
sh -c 'echo "deb [...] https://packages.sury.org/php/ $(lsb_release -sc) main" > /etc/apt/sources.list.d/php.list'
apt update -y
apt install -y php8.3 php8.3-xml php8.3-dom php8.3-gd php8.3-intl \
    php8.3-mysql php8.3-zip php8.3-curl php8.3-mbstring
apt install -y php8.3-bz2 php8.3-phar php8.3-exif php8.3-ldap openssl php8.3-opcache

# Étape 4 : Installation et sécurisation de MariaDB
apt install mariadb-server -y
mysql -e "ALTER USER 'root'@'localhost' IDENTIFIED BY 'MotDePasseGLPIFort'; FLUSH PRIVILEGES;"

# Étape 5 : Téléchargement et extraction de GLPI 10.0.20
wget https://github.com/glpi-project/glpi/releases/download/10.0.20/glpi-10.0.20.tgz
```

```

tar -xvzf glpi-10.0.20.tgz
mv glpi /var/www/

# Étape 6 : Configuration du VirtualHost Apache
a2enmod rewrite
cat <<EOF > /etc/apache2/sites-available/glpi.conf
<VirtualHost *:80>
    ServerName glpi.localhost
    ServerAlias $SERVER_IP
    DocumentRoot /var/www/glpi/public
    <Directory /var/www/glpi/public>
        Require all granted
        RewriteEngine On
        RewriteCond %{REQUEST_FILENAME} !-f
        RewriteRule ^(.*)$ index.php [QSA,L]
    </Directory>
</VirtualHost>
EOF
a2ensite glpi.conf

# Étape 7 : Permissions et sécurisation des cookies PHP
chown www-data:www-data /var/www/glpi/config/
chown -R www-data:www-data /var/www/glpi/files/
chown www-data:www-data /var/www/glpi/marketplace
sed -i 's/.*session.cookie_httponly.*/session.cookie_httponly = On/'
/etc/php/8.3/apache2/php.ini

# Étape 8 : Redémarrage Apache + message final
systemctl restart apache2
echo "=== Installation terminée ! ==="
echo "Rendez-vous sur http://$SERVER_IP pour finaliser via l'interface web."

```

Étape 2 — Sauvegarde automatique vers Google Drive (glpi-backup.sh)

J'ai mis en place un script de sauvegarde automatique qui crée un dump SQL de la base de données et une archive des fichiers GLPI, puis les transfère vers Google Drive via rclone. Les fichiers de plus de 7 jours sont automatiquement supprimés.

```

#!/bin/bash

# Horodatage pour nommer les fichiers de sauvegarde
DATE=$(date +"%Y-%m-%d_%H-%M-%S")

SOURCE_FILES="/var/www/glpi"      # Dossier source GLPI
BACKUP_DIR="/home/glpi_backup"    # Dossier local de sauvegarde
DB_NAME="glpi"                   # Nom de la base de données
CLOUD_REMOTE="google_drive"      # Nom du remote rclone
CLOUD_BUCKET="BackupsGLPI"       # Dossier sur Google Drive

mkdir -p "$BACKUP_DIR"

# Étape 1 : Dump SQL de la base de données GLPI
echo "Début du dump SQL..."
mysqldump --databases "$DB_NAME" > "$BACKUP_DIR/glpi_db_$DATE.sql"

```

```
# Étape 2 : Archive compressée des fichiers GLPI
echo "Début de la compression des fichiers..."
tar -czf "$BACKUP_DIR/glpi_files_$DATE.tar.gz" "$SOURCE_FILES"

# Étape 3 : Envoi des fichiers vers Google Drive via rclone
echo "Envoi vers le cloud..."
rclone copy "$BACKUP_DIR/glpi_db_$DATE.sql" "$CLOUD_REMOTE:$CLOUD_BUCKET"
rclone copy "$BACKUP_DIR/glpi_files_$DATE.tar.gz" "$CLOUD_REMOTE:$CLOUD_BUCKET"

# Étape 4 : Purge des sauvegardes locales de plus de 7 jours
find "$BACKUP_DIR" -type f -mtime +7 -delete

echo "Backup terminé avec succès pour le $DATE"
```

Étape 3 — Restauration depuis Google Drive (glpi-restorebackup.sh)

En cas de panne ou de perte de données, ce script récupère automatiquement la dernière sauvegarde disponible sur Google Drive, restaure la base de données et les fichiers GLPI.

```
#!/bin/bash

# Configuration (doit correspondre au script de backup)
SOURCE_FILES="/var/www/glpi"
BACKUP_DIR="/home/glpi_restore_tmp"
DB_NAME="glpi"
CLOUD_REMOTE="google_drive"
CLOUD_BUCKET="BackupsGLPI"

mkdir -p "$BACKUP_DIR"

# Étape 1 : Récupération des noms des derniers fichiers sur le Cloud
echo "--- Récupération des noms des derniers fichiers sur le Cloud ---"
LAST_SQL=$(rclone lsf "$CLOUD_REMOTE:$CLOUD_BUCKET" | grep 'glpi_db_' | sort -r | head -n 1)
LAST_FILES=$(rclone lsf "$CLOUD_REMOTE:$CLOUD_BUCKET" | grep 'glpi_files_' | sort -r | head -n 1)

if [ -z "$LAST_SQL" ] || [ -z "$LAST_FILES" ]; then
    echo "Erreur : Impossible de trouver les sauvegardes sur le cloud."
    exit 1
fi

# Étape 2 : Téléchargement de la dernière sauvegarde
echo "Téléchargement de $LAST_SQL et $LAST_FILES..."
rclone copy "$CLOUD_REMOTE:$CLOUD_BUCKET/$LAST_SQL" "$BACKUP_DIR"
rclone copy "$CLOUD_REMOTE:$CLOUD_BUCKET/$LAST_FILES" "$BACKUP_DIR"

# Étape 3 : Restauration de la base de données
echo "--- Restauration de la base de données ---"
mysql "$DB_NAME" < "$BACKUP_DIR/$LAST_SQL"
echo "Base de données restaurée."

# Étape 4 : Restauration des fichiers GLPI
echo "--- Restauration des fichiers ---"
tar -xzf "$BACKUP_DIR/$LAST_FILES" -C /
```

```
# Étape 5 : Nettoyage du dossier temporaire
echo "--- Nettoyage ---"
rm -rf "$BACKUP_DIR"

echo "Restauration terminée avec succès à partir des fichiers du $(date)"
```

Conclusion

Cette réalisation m'a permis de déployer de bout en bout un service ITSM complet en environnement cloud. J'ai acquis des compétences solides en administration Linux, configuration Apache/PHP/MariaDB, et automatisation des sauvegardes. Le service est opérationnel, accessible publiquement et sécurisé avec des sauvegardes automatiques vers Google Drive.